

How the Live Interview-Convergence Test Works

The §3.4 intake-convergence capability: its machinery, its statistics, and the gap #1c robustness layer

Model Project Constructor

2026-06-19

The Model Project Constructor pipeline begins with an automated intake interview: an agent interviews a business stakeholder and must decide, within a fixed question budget, that it has learned enough to write a structured report. *Interview convergence* is the quality property that says that decision happens reliably. This document explains the live test that measures it — how a single interview sample runs as a loop between two language models, the precise definition of “converged”, the statistics layer that makes the measurement robust, the corpus it runs on (including a negative control), and the two call sites that consume the same definition. It is written for a technical reader with introductory knowledge of large language models. Every claim is cited to a specific source file and line.

Table of contents

1	The capability and why it matters	2
1.1	The capability inside the evaluation gate	2
2	How one interview sample runs: the two-LLM loop	3
2.1	The two participants	3
2.2	The driver loop	3
2.3	Why the simulator exists	4
3	The convergence definition: the questions_cap_reached marker	4
3.1	Where the marker comes from	5
3.2	Why not gate on <code>status == "COMPLETE"</code> ? (gap #1b)	5
3.3	The premature-convergence guard	5
4	The statistics layer and the gap #1c robustness rationale	6
4.1	Why “four cases, run once” was fragile (gap #1c)	6
4.2	Two robustness fixes (the threshold left untouched)	6
4.3	The three-way outcome taxonomy	7
4.4	The result, and why it is unit-testable without an API key	8
5	The corpus, including the negative control	8
5.1	The negative-control cap fixture	9

5.2	One corpus, two tiers	9
6	Gate versus producer: the two call sites	9
6.1	The gate (asserts)	10
6.2	The producer (measures and reports)	10
6.3	Why have both	10
7	References	11
7.1	Glossary of initialisms	12

1 The capability and why it matters

The Model Project Constructor is a six-step pipeline that takes a business idea from an intake interview through data collection and validation to an initial model website, delivered as a project for a data science team. **Step 1 is the Business Intake Interview.** An agent — acting as an expert data scientist, business analyst, and consultant for a **property and casualty (P&C)** insurer’s claims organization — conducts a guided discussion with a business stakeholder. It asks **one question at a time, up to a hard cap of 20 questions**, and converges on four output sections: the business problem, the proposed solution, the model solution (target plus inputs), and the estimated value.

Everything downstream depends on this step terminating *usefully*. If the interviewer cannot decide it has enough information within the 20-question budget, it cannot produce the structured Intake Report that Steps 2 onward consume. **Interview convergence** is exactly that property: the interviewer judged it had learned enough *before* exhausting the budget.

1.1 The capability inside the evaluation gate

Convergence is one capability in a larger provider-quality gate defined in **Section 3.4 of the multi-provider large-language-model (LLM) plan** (hereafter **§3.4**). That gate exists to compare LLM providers — an **anthropic** incumbent baseline against a **bedrock** candidate — on the same golden corpus, so a provider cutover can be made on evidence rather than intuition. (**bedrock** is currently skipped because no Amazon Web Services credentials are available; the battery runs the **anthropic** half.)

The interview-convergence capability has two thresholds, single-sourced in `eval_thresholds.py`:

```
# tests/eval/eval_thresholds.py:30-31
INTERVIEW_CONVERGENCE_MIN = 0.95    # at least 95% of interviews converge
INTERVIEW_PREMATURE_MAX = 0          # zero early-convergence cases
```

i Status (as of this writing)

Convergence is **live-confirmed GREEN at 100% (20 of 20 samples)**, measured twice through independent entry points in Session 170, and the gate is **robust to network transport blips** after Session 171. The thresholds above were deliberately *never lowered* to manufacture a pass — the work described in Section 4 was a harness-statistics fix, not a threshold

loosening.

2 How one interview sample runs: the two-LLM loop

A single sample means: drive a real interviewer all the way to a terminal report, then ask whether that report shows convergence. Driving it requires a *second* language model to play the stakeholder, because a live interviewer generates its own questions and we cannot know them in advance.

2.1 The two participants

1. **The interviewer** — the intake agent under test (`IntakeAgent`). It generates each question itself and, at some point, records an internal judgment (`believe_enough_info`) that it has learned enough.
2. **The stakeholder simulator** — `StakeholderSimulator` (`tests/eval/stakeholder_sim.py`), an `AnswerProvider`. Handed *whatever question the interviewer actually asked*, it answers from the fixture’s full knowledge brief.

The simulator is deliberately wired to the **same provider, credentials, and model tier** as the interviewer under test — it reuses the interviewer’s own software development kit (SDK) client handle and resolved model identifier — so the two LLMs are matched:

```
# tests/eval/stakeholder_sim.py:153-175 (stakeholder_simulator_for)
intake_client = cast(Any, make_intake_client(provider, model=model))
return StakeholderSimulator(
    brief=_brief_from_fixture(fixture),
    client=intake_client._client,    # borrow the same SDK handle ...
    model=intake_client._model,      # ... and the same resolved model id
    max_tokens=max_tokens,
)
```

The brief it answers from is assembled from everything the stakeholder could plausibly know: the initial problem they raised, the facts established in prior discussion (`qa_pairs`), and the **complete blessed draft** — which carries the value-measurement plan the interviewer must probe to converge (`stakeholder_sim.py:69-97`). Because the simulator answers *on demand*, it can never “run out” of answers.

2.2 The driver loop

The live test builds one sample as a `run_one` callable:

```
# tests/eval/test_eval_live.py:140-150 (inside test_live_interview_converges)
def run_one(case: InterviewCase) -> IntakeReport:
    fixture = load_fixture(case.fixture_path)
    agent = IntakeAgent(llm=make_intake_client(provider))
    return agent.run_scripted(
```

```

    stakeholder_id=fixture["stakeholder_id"],
    session_id=f"live-{{fixture['session_id']}}",
    domain=fixture.get("domain", "pc_claims"),
    initial_problem=fixture.get("initial_problem"),
    answer_provider=stakeholder_simulator_for(fixture, provider=provider),
    review_responses=review_sequence_from_fixture(fixture),
)

```

`run_scripted` (`agent.py:68`) drives the compiled interview graph (a `LangGraph` state machine) headlessly. It starts the graph, then services each *interrupt* the graph raises until the graph's `finalize` step completes:

```

graph.invoke(initial_state)
repeat (bounded by max_turns = MAX_QUESTIONS + MAX_REVISIONS + 5 = 28):
    snapshot = graph.get_state()
    if no pending interrupt: break
    if interrupt is a "question":
        reply = answer_provider(question, question_number) # the simulator
    elif interrupt is a "review":
        reply = next(review_responses) # a fixed list
    graph.invoke(resume = reply)
return build_intake_report(final_state)

```

Two answer sources are possible, and exactly one must be supplied (`agent.py:81-98`): a fixed `interview_answers` list (used by the deterministic tier), or an `answer_provider` (used by the live tier). The `max_turns` bound is the only thing standing between a buggy graph and an infinite loop (`agent.py:108-160`).

2.3 Why the simulator exists

The original deterministic approach replayed a *fixed recorded list* of answers, one per recorded question. That only works when the questions are fixed. A live interviewer asks more or different questions than any recorded script supplies, so the list ran out — `run_scripted` then raised `RuntimeError("Fixture ran out of interview answers ...")` and the interview was scored as a non-convergence. That was a **harness artifact**, not model behavior (recorded in `PROJECT_LEARNINGS #21` and the §3.4 agreement report).

The simulator (Session 166) closed that hole. Because it answers any question on demand from the fixture's full knowledge, the script can no longer starve, so **a sub-convergence now reflects the interviewer model's own behavior** (`stakeholder_sim.py:1-23`).

3 The convergence definition: the `questions_cap_reached` marker

Convergence is decided by a single, deliberately narrow predicate:

```
# tests/eval/eval_scoring.py:92-107 (interview_converged)
def interview_converged(report: IntakeReport) -> bool:
    return "questions_cap_reached" not in report.missing_fields
```

So convergence is defined by the **absence** of one marker string in the report’s `missing_fields`. To trust that definition you have to know exactly when that marker is added.

3.1 Where the marker comes from

The interview graph’s `finalize` step appends it under one precise condition:

```
# src/model_project_constructor/agents/intake/nodes.py:131-136 (finalize_node)
if (
    asked >= MAX_QUESTIONS          # MAX_QUESTIONS == 20
    and not state.get("believe_enough_info", False)
    and "questions_cap_reached" not in missing
):
    missing.append("questions_cap_reached")
```

Reading the condition: the marker is present **if and only if** the interview hit the 20-question cap *without* the interviewer ever recording `believe_enough_info`. Therefore its **absence** is precisely “the interviewer believed it had enough information within the cap” — which is exactly the §3.4 metric text. (`believe_enough_info` is the interviewer LLM’s own judgment, recorded into graph state during the drafting decision, `nodes.py:58, 82.`)

3.2 Why not gate on `status == "COMPLETE"`? (gap #1b)

It is tempting to call an interview “converged” when its report is `COMPLETE`. That is wrong, and fixing it was **gap #1b** (Session 168). A report is `COMPLETE` only when the human review was accepted *and* there are zero missing fields (`nodes.py:154`) — that is report **finalization**, a stricter and *different* property than interview convergence. Gating on `COMPLETE` measured report completeness, not whether the interview converged; the scorer was stricter than its own §3.4 metric. Keying on the marker fixed the mismatch (`eval_scoring.py:99-105`).

3.3 The premature-convergence guard

The §3.4 metric also requires “zero premature convergence”. A companion scorer flags an interview that converged *too early*:

```
# tests/eval/eval_scoring.py:110-126 (premature_convergence)
return (
    "questions_cap_reached" not in report.missing_fields # same convergence signal
    and report.questions_asked < min_questions           # default = 2
)
```

Critically, this guard keys on the **same** convergence signal as `interview_converged`, not on `status == "COMPLETE"`. If it keyed on `COMPLETE`, an interview could converge at question 1 yet never finalize and so slip past the premature check (`eval_scoring.py:117-121`).

4 The statistics layer and the gap #1c robustness rationale

4.1 Why “four cases, run once” was fragile (gap #1c)

Originally, both call sites ran each interview fixture **exactly once** and computed a pass rate over the resulting booleans against the 95% bar. There are four converging fixtures (Section 5), so the rate was a fraction over four samples. **95% over four samples mathematically requires 4 out of 4**, because 3 out of 4 is only 75%, which is below 95%.

That made the gate brutally fragile. A *single* miss — whether the model genuinely wobbled once, or a one-off transient blip occurred (a truncated application programming interface (API) response, an empty simulated answer, a network timeout) — dropped the gate to 75% and **failed the entire roughly 90-minute live run**. The gate was measuring luck, not capability (`interview_sweep.py:11-15`).

4.2 Two robustness fixes (the threshold left untouched)

`interview_sweep.py` owns the statistics of the measurement so the assertion gate and the report producer stay in lock-step on one definition. It applies two fixes, and explicitly **does not move the 95% bar** — this is a harness-statistics fix, not a threshold loosening.

Fix 1 — sample each case at least five times and pool the results.

```
# tests/eval/interview_sweep.py:64
N_SAMPLES = 5
```

This matches the discipline the governance live test already used. With five samples per case across four cases, the pool is 20 samples, so one transient miss is 1 in 20 (95%), not 1 in 4 (75%) — it no longer sinks the gate (`interview_sweep.py:111-149`).

Fix 2 — tell a transient blip apart from a genuine non-convergence. A once-off failure that is an artifact of *the call* — not the interviewer declining to converge — is retried a bounded number of times and, if it still fails, **excluded** from the sample with a logged note rather than scored as a non-convergence:

```
# tests/eval/interview_sweep.py:68
_MAX_TRANSIENT_RETRIES = 2 # so up to 3 attempts per sample before exclusion
```

The classification logic lives in `_run_one_sample` (`interview_sweep.py:152-180`): it retries on a transient error, and returns `None` (the sample is excluded) only when every attempt hit a transient error. A non-transient error is *not* caught — it propagates.

4.3 The three-way outcome taxonomy

The heart of the robustness layer is a deliberate three-way split. Every sample ends in exactly one of these outcomes:

Table 1: Every interview sample ends in exactly one of three outcomes.

Outcome	What it is	Examples	Handling
Transient	An artifact of the API / seam / transport, not interview behavior	<code>IntakeLLMError</code> (intake client got a truncated/empty/malformed response); <code>StakeholderSimError</code> (simulator returned empty content); <code>APITimeoutError</code> / <code>APIConnectionError</code> (Anthropic SDK network blip, raised only <i>after</i> the SDK exhausts its own retries)	Retried up to 2 times; if still failing, the sample is excluded (counted separately, with a logged note) and never scored as a miss.
Genuine non-convergence	The interviewer returned a report carrying <code>questions_cap_reached</code>	The graph self-terminates at the 20-question cap <i>with a report</i> — it does not raise	Scored False — it counts against the rate, as it should.
Real bug	An API / harness / graph defect, not a model miss	<code>APIStatusError</code> (4xx/5xx: bad model id, auth, rate limit) — a <i>sibling</i> of <code>APIConnectionError</code> , deliberately not caught; a non-seam <code>RuntimeError</code> (unknown interrupt, starved review script, the max-turns backstop)	Propagates and aborts the run. Silently scoring these False is what once hid harness defects as model misses.

The transient set is single-sourced:

```
# tests/eval/interview_sweep.py:82-87
_TRANSIENT_ERRORS: tuple[type[Exception], ...] = (
    IntakeLLMError,          # application/seam: subclass of RuntimeError
    StakeholderSimError,     # application/seam: subclass of RuntimeError
    APITimeoutError,         # transport: subclass of APIConnectionError
    APIConnectionError,     # transport
)
```

! The transport half was a residual (gap #1c, Session 171)

A transient classifier is only complete if it covers **both** application/seam errors **and** transport errors. The original tuple held only the first two; a live gate run then aborted on a raw `APITimeoutError` (a Secure Sockets Layer handshake timeout) that propagated past it.

Session 171 added the two transport classes. The boundary is intentional: `APIStatusError` (a genuine 4xx/5xx API error) is a *sibling* of `APIConnectionError`, **not** a subclass, so it is not caught and still surfaces loudly — guarding against the failure mode (project “FM #18”) of a catch silently swallowing real bugs (`interview_sweep.py:70-87`).

4.4 The result, and why it is unit-testable without an API key

The sweep returns a small result object (`interview_sweep.py:96-109`):

```
@dataclass(frozen=True)
class InterviewSweepResult:
    convergence_results: list[bool]    # pooled per-sample pass/fail
    premature_count: int               # §3.4 premature-convergence count
    excluded_transient: int            # samples dropped after retries
```

The sweep itself **calls no LLM**. It receives a `run_one` callable that performs one live interview, so the sampling and classification logic is deterministically unit-tested with a fake runner (`test_interview_sweep.py`) — no API key, no network — mirroring how `eval_scoring.py` keeps scoring testable (`interview_sweep.py:42-46`). The Session 171 transport-error addition was verified this way: four deterministic tests built from genuine SDK exception objects, with no live run.

5 The corpus, including the negative control

The interview goldens are declared in `eval_corpus.py` and loaded by `load_interview_cases()`:

```
# tests/eval/eval_corpus.py:103-109  (_INTERVIEW_SOURCES)
_INTERVIEW_SOURCES = [
    ("subrogation",          .../subrogation.yaml,          True),
    ("pricing_optimization", .../pricing_optimization.yaml, True),
    ("fraud_triage",         .../fraud_triage.yaml,         True),
    ("reserving_adequacy",   .../reserving_adequacy.yaml,   True),
    ("question_cap",        .../intake_question_cap.yaml,   False), # <- control
]
```

Table 2: The five interview goldens; the fixture paths are listed in the code block above.

case_id	expect_complete	Role
subrogation	True	converging — reused project fixture
pricing_optimization	True	converging — reused project fixture
fraud_triage	True	converging — reused project fixture
reserving_adequacy	True	converging — authored for the eval
question_cap	False	negative control (terminates at the cap)

Three of the four converging cases reuse the project’s own blessed intake fixtures, which keeps the goldens single-sourced rather than duplicated (`eval_corpus.py:1-17`).

5.1 The negative-control cap fixture

`intake_question_cap.yaml` (`expect_complete = False`) is the inverse of a converging case. It is engineered so the interviewer *should* spend the full 20-question budget **without** ever believing it has enough — that is, it should **produce** the `questions_cap_reached` marker rather than avoid it.

The live convergence sweep deliberately **skips** it, so it never dilutes the 95% rate:

```
# tests/eval/interview_sweep.py:132-134 (sweep_interview_convergence)
for case in cases:
    if not case.expect_complete:
        continue          # the question-cap golden is not expected to converge
```

Its cap behavior is instead pinned by the **deterministic** tier, which asserts the full inverse property:

```
# tests/eval/test_eval_corpus.py:117-123 (test_interview_question_cap_terminates)
assert not interview_converged(report)
assert report.status == "DRAFT_INCOMPLETE"
assert "questions_cap_reached" in report.missing_fields
assert report.questions_asked == MAX_QUESTIONS      # == 20
```

(The same cap behavior is also covered at the graph level in `test_graph.py`.)

Why a negative control matters. A convergence metric that only ever sees cases that *should* converge cannot distinguish “the agent converges correctly” from “the agent always says yes.” The cap fixture proves the marker actually fires when it should — that the convergence definition has teeth. This is the classic negative-control discipline applied to an LLM evaluation.

5.2 One corpus, two tiers

The same corpus feeds two test tiers. The **deterministic** tier (`test_eval_corpus.py`) replays the fixtures with a `FixtureLLMClient` (no API key) to prove the scorers and corpus are well-formed. The **live** tier (Section 6) runs a real provider plus the simulator. Same goldens, different source of model behavior.

6 Gate versus producer: the two call sites

Both consumers of the corpus call the *same* helper, `interview_sweep.sweep_interview_convergence`, so they can never drift on the definition of convergence. They differ only in stance.

6.1 The gate (asserts)

`test_live_interview_converges` is the assertion gate (`test_eval_live.py:134-157`):

```
sweep = sweep_interview_convergence(load_interview_cases(), run_one)
rate = pass_rate("interview", sweep.convergence_results)
assert rate.meets(thresholds.INTERVIEW_CONVERGENCE_MIN), (
    f"interview convergence {rate.rate:.1%} < "
    f"{thresholds.INTERVIEW_CONVERGENCE_MIN:.0%} "
    f"(n={rate.total}, excluded_transient={sweep.excluded_transient})"
)
```

It runs under `pytest -m live`, is parametrized over every shadow provider (currently three), and is auto-skipped per provider when that provider’s credentials are absent (so continuous integration (CI), which has no credentials, stays hermetic). A sub-threshold result **fails the run**. The failure message reports both the sample count and how many samples were excluded as transient.

6.2 The producer (measures and reports)

`shadow_run.measure_provider` is the report producer (`shadow_run.py:73-153`). It measures *every* §3.4 capability — including the quality-check (QC) structural metric and the Structured Query Language (SQL) metrics — and prints the per-capability rates plus a rendered agreement report. It calls the same sweep:

```
# tests/eval/shadow_run.py:139-141
sweep = sweep_interview_convergence(
    load_interview_cases(), _run_interview, n_samples=n_samples, on_event=_warn
)
```

Crucially, the producer **does not assert**. A sub-threshold number is recorded as *data* (a no-go signal), not a failed run, so it can fill the cutover report and compare providers side by side. The committed go/no-go lives in `PHASE_E_AGREEMENT_REPORT.md`, fed by `eval_cutover.evaluate_cutover` (`shadow_run.py:1-22, 156-183`). It is run explicitly, for example:

```
ANTHROPIC_API_KEY=... uv run python tests/eval/shadow_run.py
# (run from the repo root, or set PYTHONPATH=.)
```

6.3 Why have both

An assertion gate is binary — ideal for CI, useless for filling a report. A producer yields the actual numbers and must *not* “fail” merely because a candidate is sub-threshold (that is data, not an error). Same definition, two stances.

💡 The two-draw insight (Session 170)

Because both entry points share the sampling helper, one `shadow_run` is the gate’s convergence verdict. But running **two independent draws through different entry points** is not redundant — it tests robustness *across* runs, which is the very concern of a fragility fix. It was the second draw (the gate) that surfaced the uncaught transport-timeout residual closed in Section 4. A subject-matter expert (SME) reviewing such a gate should expect the second draw to earn its keep.

7 References

The six files this document explains (all relative to `tests/eval/`), plus the supporting machinery it cites.

Table 3: The six cited files (paths relative to `tests/eval/`).

File	Role	Key symbols (lines)
<code>test_eval_live.py</code>	The assertion gate (live tier)	<code>test_live_interview_converges</code> (134-157)
<code>interview_sweep.py</code>	The statistics layer: sampling and transient classification	<code>sweep_interview_convergence</code> (111-149); <code>_run_one_sample</code> (152-180); <code>_TRANSIENT_ERRORS</code> (82-87); <code>N_SAMPLES</code> (64)
<code>eval_scoring.py</code>	The convergence definition	<code>interview_converged</code> (92-107); <code>premature_convergence</code> (110-126)
<code>eval_corpus.py</code>	The corpus loaders	<code>_INTERVIEW_SOURCES</code> (103-109); <code>load_interview_cases</code> (159-161)
<code>shadow_run.py</code>	The measurement producer	<code>measure_provider</code> (73-153)
<code>eval_thresholds.py</code>	The thresholds	<code>INTERVIEW_CONVERGENCE_MIN</code> , <code>INTERVIEW_PREMATURE_MAX</code> (30-31)

Supporting files cited above (the two `src` files live under `src/model_project_constructor/agents/intake/`):

Table 4: Supporting files cited in this document.

File	Why it appears here
<code>stakeholder_sim.py</code>	The second LLM: the on-demand stakeholder simulator (Section 2)
<code>agent.py</code>	<code>run_scripted</code> , the driver loop (Section 2)
<code>nodes.py</code>	<code>finalize_node</code> , where <code>questions_cap_reached</code> is set (Section 3)
<code>test_eval_corpus.py</code>	The deterministic tier; pins the negative control (Section 5.1)

7.1 Glossary of initialisms

API application programming interface

CI continuous integration

LLM large language model

P&C property and casualty (insurance)

QC quality check

SDK software development kit

SME subject-matter expert

SQL Structured Query Language

§3.4 Section 3.4 of the multi-provider LLM plan (the provider-quality gate)